Monitorando a Comunicação de Aplicações via RabbitMQ: Como o OpenTelemetry Consegue Isso?

Autor: Manus AI

Data: Março de 2026

Versão: 1.0.0

Sumário Executivo

Este material apresenta um guia prático e completo sobre como instrumentar aplicações NestJS para monitorar a comunicação entre microsserviços através do RabbitMQ utilizando OpenTelemetry. O rastreamento distribuído permite que você visualize o caminho completo de uma mensagem desde sua criação no produtor até seu processamento no consumidor, incluindo latências, erros e contexto de execução em tempo real.

Você aprenderá os conceitos fundamentais de observabilidade, como implementar a instrumentação automática do RabbitMQ, e como visualizar os rastros (traces) em uma interface gráfica usando Jaeger. Ao final, terá dois microserviços totalmente funcionais que demonstram essas práticas em ação.

1. Introdução à Observabilidade e Rastreamento Distribuído

1.1 O Problema da Visibilidade em Microsserviços

Quando você trabalha com uma arquitetura de microsserviços, a comunicação entre serviços torna-se complexa. Uma simples requisição do usuário pode percorrer múltiplos serviços, bancos de dados e filas de mensagens. Sem ferramentas adequadas, é extremamente difícil entender:

- **Onde** uma mensagem está sendo processada
- **Quanto tempo** leva para ser entregue e processada
- **Se** houve erros em algum ponto da jornada
- **Qual** é o contexto de execução em cada etapa

Este é o desafio que a observabilidade moderna resolve.

1.2 Os Três Pilares da Observabilidade

A observabilidade moderna é construída sobre três pilares fundamentais:

Pilar	Descrição	Exemplo
Traces (Rastros)	Registram o caminho completo de uma requisição através de múltiplos serviços	Uma mensagem enviada ao RabbitMQ e processada por um consumidor
Metrics (Métricas)	Medem valores agregados sobre o comportamento do sistema	Número de mensagens processadas por segundo, latência média
Logs (Registros)	Registram eventos discretos com contexto detalhado	"Pedido #123 recebido", "Erro ao processar pedido"

Este material foca principalmente em **traces**, que são essenciais para entender o fluxo de mensagens no RabbitMQ.

1.3 O que é OpenTelemetry?

OpenTelemetry é uma especificação e conjunto de bibliotecas de código aberto que padronizam como capturar sinais de observabilidade (traces, métricas e logs) em aplicações. Ele fornece:

- **Instrumentação automática** para bibliotecas populares (como amqplib para RabbitMQ)
- **Propagação de contexto** entre serviços para correlacionar eventos
- **Exportadores** para enviar dados para backends de observabilidade (Jaeger, Datadog, etc.)
- **SDK** para criar instrumentação customizada

---

2. Conceitos Fundamentais do Rastreamento Distribuído

2.1 Spans e Traces

Um **trace** é uma representação da jornada completa de uma requisição através de seu sistema. Um **span** é uma unidade individual de trabalho dentro desse trace.

```
Plain Text
```

```
Trace ID: abc123def456
├── Span: POST /orders (produtor)
│   ├── Span: emit('order.created', order)
│   └── Span: envio para RabbitMQ
└── Span: handleOrderCreated (consumidor)
    ├── Span: recebimento da fila
    └── Span: processamento do pedido
```

Cada span contém:

- **Trace ID**: Identificador único que conecta todos os spans de uma requisição
- **Span ID**: Identificador único do span individual
- **Parent Span ID**: Referência ao span pai (se houver)
- **Timestamps**: Hora de início e fim
- **Duração**: Tempo total de execução
- **Atributos**: Dados contextuais (ex: ID do pedido, ID do cliente)
- **Eventos**: Marcos importantes durante a execução
- **Status**: Sucesso, erro ou não definido

2.2 Propagação de Contexto

A propagação de contexto é o mecanismo que permite que o Trace ID seja passado de um serviço para outro através de headers HTTP, cabeçalhos de mensagem ou outros mecanismos de transporte.

No caso do RabbitMQ, o OpenTelemetry injeta automaticamente o contexto nos headers da mensagem:

JSON

```
{
  "traceparent": "00-abc123def456-def456abc123-01",
  "tracestate": "..."
}
```

Quando o consumidor recebe a mensagem, extrai esse contexto e continua o trace com o mesmo Trace ID, criando uma cadeia visual de eventos.

2.3 Instrumentação Automática vs. Manual

Instrumentação Automática: O OpenTelemetry detecta automaticamente chamadas para bibliotecas conhecidas (como amqplib) e cria spans sem modificar seu código.

Instrumentação Manual: Você cria spans explicitamente em seu código para capturar lógica de negócio customizada.

Neste material, usamos ambas as abordagens:

- **Automática:** Para operações de RabbitMQ (envio e recebimento de mensagens)
- **Manual:** Para lógica de negócio (criação de pedidos, processamento)

---

3. Arquitetura da Solução

3.1 Componentes da Stack

Nossa solução utiliza os seguintes componentes:

Componente	Versão	Propósito
NestJS	10.3.0	Framework para construir os microsserviços
RabbitMQ	3.12	Message broker para comunicação assíncrona
OpenTelemetry SDK	0.50.0	SDK Node.js para coleta de traces
OpenTelemetry Collector	latest	Agregador de traces
Jaeger	latest	Backend de visualização de traces

3.2 Fluxo de Dados

Plain Text

```
┌──────────────────────────────────────────────────────────────┐
│                    Aplicação Produtor                        │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ Requisição HTTP POST /orders                         │    │
│  │ ↓                                                    │    │
│  │ OrderService.createOrder()                           │    │
│  │ ├─ Criar span "createOrder"                          │    │
│  │ ├─ Enviar mensagem para RabbitMQ                     │    │
│  │ │  (OpenTelemetry injeta contexto automaticamente)   │    │
│  │ └─ Registrar eventos no span                         │    │
```



4. Configuração do Ambiente

4.1 Pré-requisitos

Antes de começar, certifique-se de que você tem instalado:

- **Node.js** 16.0.0 ou superior
- **npm** ou **yarn**
- **Docker** e **Docker Compose**

- **Git** (opcional, mas recomendado)

4.2 Estrutura de Diretórios

Plain Text

```
nestjs-otel-rabbitmq/
├── producer/
│   ├── src/
│   │   ├── main.ts
│   │   ├── tracing.ts
│   │   ├── app.module.ts
│   │   ├── order.service.ts
│   │   ├── order.controller.ts
│   │   └── rabbitmq.config.ts
│   ├── package.json
│   ├── tsconfig.json
│   └── README.md
├── consumer/
│   ├── src/
│   │   ├── main.ts
│   │   ├── tracing.ts
│   │   ├── app.module.ts
│   │   ├── order.consumer.ts
│   │   └── rabbitmq.config.ts
│   ├── package.json
│   ├── tsconfig.json
│   └── README.md
└── docker/
    ├── docker-compose.yml
    └── otel-collector-config.yml
```

4.3 Iniciando os Serviços de Infraestrutura

Navegue até o diretório `docker` e execute:

Bash

```
cd docker
docker-compose up -d
```

Isso iniciará:

- **RabbitMQ** em `http://localhost:15672` (usuário: guest, senha: guest)
- **Jaeger** em `http://localhost:16686`

- **OpenTelemetry Collector** em `http://localhost:4318`

Verifique se os serviços estão rodando:

```bash
docker-compose ps
```

---

5. Implementação Passo a Passo

5.1 Configuração do OpenTelemetry

O arquivo `tracing.ts` é o ponto de entrada para a instrumentação. Ele deve ser importado **primeiro** na aplicação, antes de qualquer outro código:

```typescript
// src/main.ts
import './tracing'; // ← DEVE SER A PRIMEIRA LINHA
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
```

Por que isso é importante? O OpenTelemetry precisa interceptar as chamadas de biblioteca antes que elas ocorram. Se importado depois, ele não conseguirá instrumentar corretamente.

5.1.1 Estrutura do Arquivo tracing.ts

```typescript
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentatio
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';
import { Resource } from '@opentelemetry/resources';
import { SemanticResourceAttributes } from '@opentelemetry/semantic-conventions
import { AmqplibInstrumentation } from '@opentelemetry/instrumentation-amqplib'

// 1. Criar exportador que envia traces para o Collector
const traceExporter = new OTLPTraceExporter({
  url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT || 'http://localhost:4318/v1/tra
} );

// 2. Configurar o SDK com recurso (identificação do serviço)
const sdk = new NodeSDK({
```

```
  resource: new Resource({
    [SemanticResourceAttributes.SERVICE_NAME]: 'producer-service',
    [SemanticResourceAttributes.SERVICE_VERSION]: '1.0.0',
  }),
  traceExporter,
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-amqplib': {
        enabled: true,
      },
    }),
    new AmqplibInstrumentation(),
  ],
});

// 3. Iniciar o SDK
sdk.start();

// 4. Graceful shutdown
process.on('SIGTERM', () => {
  sdk.shutdown()
    .then(() => console.log('Tracing terminated'))
    .catch((log) => console.log('Error terminating tracing', log))
    .finally(() => process.exit(0));
});
```

O que cada parte faz:

- **OTLPTraceExporter**: Configura para onde enviar os traces (Jaeger/Collector)
- **NodeSDK**: Inicializa o SDK com configurações globais
- **Resource**: Define metadados sobre o serviço (nome, versão)
- **Instrumentations**: Ativa a coleta automática de dados de bibliotecas específicas
- **AmqplibInstrumentation**: Instrumentação específica para RabbitMQ

5.2 Configuração do RabbitMQ

O arquivo `rabbitmq.config.ts` centraliza a configuração de conexão:

TypeScript

```
import { Transport, ClientOptions } from '@nestjs/microservices';

export const rabbitmqConfig: ClientOptions = {
  transport: Transport.RMQ,
  options: {
    urls: [process.env.RABBITMQ_URL || 'amqp://guest:guest@localhost:5672'],
```

```
    queue: 'orders_queue',
    queueOptions: {
      durable: true, // Fila persiste mesmo se o RabbitMQ reiniciar
    },
    prefetchCount: 1, // Consumidor processa 1 mensagem por vez
  },
};
```

Explicação dos parâmetros:

- **urls**: Endereço do RabbitMQ
- **queue**: Nome da fila onde as mensagens serão armazenadas
- **durable**: Se true, a fila não é deletada quando o RabbitMQ reinicia
- **prefetchCount**: Quantas mensagens o consumidor busca por vez

5.3 Serviço Produtor (OrderService)

O serviço produtor cria pedidos e os envia para o RabbitMQ:

TypeScript

```
import { Injectable, Inject } from '@nestjs/common';
import { ClientProxy } from '@nestjs/microservices';
import { trace } from '@opentelemetry/api';

const tracer = trace.getTracer('producer-service');

@Injectable()
export class OrderService {
  constructor(@Inject('ORDER_SERVICE') private client: ClientProxy) {}

  async createOrder(order: Omit<Order, 'id' | 'timestamp'>): Promise<Order> {
    // 1. Criar um span para esta operação
    const span = tracer.startSpan('createOrder');

    try {
      // 2. Criar o objeto de pedido
      const newOrder: Order = {
        ...order,
        id: `ORD-${Date.now()}`,
        timestamp: new Date(),
      };

      // 3. Registrar um evento no span com dados do pedido
      span.addEvent('order_created', {
        'order.id': newOrder.id,
```

```
        'order.customer_id': newOrder.customerId,
        'order.amount': newOrder.amount,
      });

      // 4. Enviar para RabbitMQ
      // OpenTelemetry injeta automaticamente o contexto nos headers
      await this.client.emit('order.created', newOrder).toPromise();

      // 5. Registrar sucesso
      span.addEvent('order_sent_to_queue', {
        'queue.name': 'orders_queue',
      });

      return newOrder;
    } catch (error) {
      // 6. Registrar erro no span
      span.recordException(error as Error);
      throw error;
    } finally {
      // 7. Finalizar o span
      span.end();
    }
  }
}
```

Pontos-chave:

- **tracer.startSpan()**: Cria um novo span
- **span.addEvent()**: Registra eventos importantes durante a execução
- **span.recordException()**: Registra exceções
- **span.end()**: Finaliza o span (obrigatório)

5.4 Controlador Produtor (OrderController)

O controlador expõe endpoints HTTP:

TypeScript

```
import { Controller, Post, Get, Body } from '@nestjs/common';
import { OrderService, Order } from './order.service';

@Controller('orders')
export class OrderController {
  constructor(private readonly orderService: OrderService) {}

  @Post()
```

```
  async createOrder(
    @Body() order: Omit<Order, 'id' | 'timestamp'>,
  ): Promise<Order> {
    return this.orderService.createOrder(order);
  }

  @Get()
  async getOrders(): Promise<Order[]> {
    return this.orderService.getOrders();
  }
}
```

5.5 Consumidor (OrderConsumer)

O consumidor escuta a fila e processa mensagens:

TypeScript

```
import { Controller } from '@nestjs/common';
import { EventPattern } from '@nestjs/microservices';
import { trace } from '@opentelemetry/api';

const tracer = trace.getTracer('consumer-service');

@Controller()
export class OrderConsumer {
  @EventPattern('order.created')
  async handleOrderCreated(order: Order) {
    // 1. Criar um span para o processamento
    const span = tracer.startSpan('handleOrderCreated');

    try {
      // 2. Registrar recebimento
      span.addEvent('order_received', {
        'order.id': order.id,
        'order.customer_id': order.customerId,
        'order.amount': order.amount,
      });

      // 3. Processar o pedido
      console.log(`Processing order: ${order.id}`);
      await new Promise((resolve) => setTimeout(resolve, 500));

      // 4. Registrar sucesso
      span.addEvent('order_processed', {
        'order.id': order.id,
        'processing.duration_ms': 500,
```

```
      });

      console.log(`Order ${order.id} processed successfully`);
    } catch (error) {
      // 5. Registrar erro
      span.recordException(error as Error);
      console.error(`Error processing order ${order.id}:`, error);
    } finally {
      // 6. Finalizar o span
      span.end();
    }
  }
}
```

Importante: O OpenTelemetry extrai automaticamente o contexto de trace dos headers da mensagem, então o span criado aqui será automaticamente vinculado ao span do produtor com o mesmo Trace ID.

---

6. Instalação e Execução

6.1 Instalar Dependências

Para o produtor:

```
Bash

cd producer
npm install
```

Para o consumidor:

```
Bash

cd consumer
npm install
```

6.2 Compilar TypeScript

Para o produtor:

```
Bash

cd producer
```

```
npm run build
```

Para o consumidor:

Bash

```
cd consumer
npm run build
```

6.3 Executar em Desenvolvimento

Terminal 1 - Produtor:

Bash

```
cd producer
npm run dev
```

Você verá:

Plain Text

```
[Nest]: # "12345  - 03/10/2026, 10:30:00 AM     LOG [NestFactory] Starting Nest
[Nest]: # "12345  - 03/10/2026, 10:30:01 AM     LOG [InstanceLoader] AppModule
Tracing initialized for producer service
Producer service is running on http://localhost:3001
```

Terminal 2 - Consumidor:

Bash

```
cd consumer
npm run dev
```

Você verá:

Plain Text

```
[Nest]: # "12346  - 03/10/2026, 10:30:05 AM     LOG [NestFactory] Starting Nest
[Nest]: # "12346  - 03/10/2026, 10:30:06 AM     LOG [InstanceLoader] AppModule
Tracing initialized for consumer service
Consumer service is listening to RabbitMQ
```

6.4 Testar a Comunicação

Terminal 3 - Criar um pedido:

```bash
curl -X POST http://localhost:3001/orders \
  -H "Content-Type: application/json" \
  -d '{
    "customerId": "CUST-001",
    "amount": 100.50,
    "items": ["item1", "item2"]
  }'
```

Você verá a resposta:

```json
{
  "id": "ORD-1710073800000",
  "customerId": "CUST-001",
  "amount": 100.50,
  "items": ["item1", "item2"],
  "timestamp": "2026-03-10T13:30:00.000Z"
}
```

E no terminal do consumidor:

```
Processing order: ORD-1710073800000
Order ORD-1710073800000 processed successfully
```

---

7. Visualizando os Traces no Jaeger

7.1 Acessar o Jaeger

Abra seu navegador e acesse: **http://localhost:16686**

Você verá a interface do Jaeger com um painel de busca.

7.2 Buscar Traces

1. No dropdown **Service**, selecione **producer-service**
2. Clique em **Find Traces**

Você verá uma lista de traces. Clique em um deles para visualizar os detalhes.

7.3 Entender o Gráfico de Traces

O gráfico mostra:

```
Plain Text

Trace ID: abc123def456

├─ producer-service (3.2ms )
│  └─ POST /orders (2.1ms)
│     └─ createOrder (1.8ms)
│        ├─ order_created (event)
│        ├─ emit to RabbitMQ (0.5ms)
│        └─ order_sent_to_queue (event)
│
└─ consumer-service (1.5ms)
   └─ handleOrderCreated (1.4ms)
      ├─ order_received (event)
      ├─ process order (0.8ms)
      └─ order_processed (event)
```

Cada linha representa um span. A indentação mostra a hierarquia (parent-child). O tempo mostra a duração de cada operação.

7.4 Analisar Dados de Trace

Clique em um span individual para ver:

- **Trace ID**: Identificador único da requisição completa
- **Span ID**: Identificador do span específico
- **Parent Span ID**: Referência ao span pai
- **Duração**: Tempo total de execução
- **Atributos**: Dados contextuais (ex: order.id, order.amount)
- **Eventos**: Marcos registrados durante a execução
- **Status**: Sucesso ou erro

---

8. Conceitos Avançados

8.1 Propagação de Contexto em Detalhes

Quando o produtor envia uma mensagem para o RabbitMQ, o OpenTelemetry injeta automaticamente o contexto nos headers:

JSON

```
{
  "traceparent": "00-abc123def456-def456abc123-01",
  "tracestate": "vendor=value"
}
```

O formato `traceparent` segue o padrão W3C Trace Context:

Plain Text

```
00 - versão
abc123def456 - Trace ID (128 bits em hex)
def456abc123 - Span ID (64 bits em hex)
01 - flags (01 = trace é amostrado)
```

Quando o consumidor recebe a mensagem, extrai esses valores e cria um novo span com o mesmo Trace ID, vinculando as operações.

8.2 Amostragem (Sampling)

Em produção, você pode não querer rastrear 100% das requisições. O OpenTelemetry suporta amostragem:

TypeScript

```
import { ProbabilitySampler } from '@opentelemetry/sdk-trace-node';

const sdk = new NodeSDK({
  // ... outras configurações
  sampler: new ProbabilitySampler(0.1), // Rastrear 10% das requisições
});
```

8.3 Baggage (Contexto Customizado)

Você pode adicionar dados customizados ao contexto que serão propagados automaticamente:

TypeScript

```
import { context, propagation, baggage } from '@opentelemetry/api';
```

```
const ctx = baggage.setBaggage(
  context.active(),
  baggage.createBaggage({
    'user.id': { value: 'user123' },
    'request.id': { value: 'req456' },
  }),
);

context.with(ctx, () => {
  // Seu código aqui
});
```

8.4 Métricas

Além de traces, você pode coletar métricas:

TypeScript

```
import { MeterProvider } from '@opentelemetry/sdk-metrics';

const meterProvider = new MeterProvider();
const meter = meterProvider.getMeter('producer-service');

const orderCounter = meter.createCounter('orders_created', {
  description: 'Total de pedidos criados',
});

orderCounter.add(1, {
  'order.status': 'created',
});
```

---

9. Troubleshooting

9.1 Problema: Traces não aparecem no Jaeger

Possíveis causas:

1. **OpenTelemetry Collector não está rodando**

   Bash

   ```
   docker-compose ps
   ```

   Se o collector não está rodando, reinicie:

Bash

```
docker-compose restart otel-collector
```

2. **Endpoint OTLP incorreto**

   Verifique se a variável de ambiente está correta:

   Bash

   ```
   echo $OTEL_EXPORTER_OTLP_ENDPOINT
   # Deve ser: http://localhost:4318/v1/traces
   ```

3. **RabbitMQ não está acessível**

   Bash

   ```
   docker-compose logs rabbitmq
   ```

9.2 Problema: Erro "Cannot find module '@opentelemetry/...'"

Solução:

Bash

```
npm install
npm run build
```

9.3 Problema: Spans não estão vinculados entre produtor e consumidor

Possível causa: A propagação de contexto não está funcionando.

Solução: Certifique-se de que ambos os serviços têm a instrumentação do amqplib ativada:

TypeScript

```
instrumentations: [
  getNodeAutoInstrumentations({
    '@opentelemetry/instrumentation-amqplib': {
      enabled: true,
    },
  } ),
  new AmqplibInstrumentation(),
],
```

10. Boas Práticas

10.1 Nomeação de Spans

Use nomes descritivos que indicam a operação:

TypeScript

```typescript
// ✅ Bom
const span = tracer.startSpan('processPayment');

// ❌ Ruim
const span = tracer.startSpan('process');
```

10.2 Registrar Dados Relevantes

Adicione atributos que ajudem na análise:

TypeScript

```typescript
// ✅ Bom
span.addEvent('payment_processed', {
  'payment.id': paymentId,
  'payment.amount': amount,
  'payment.currency': 'USD',
  'payment.method': 'credit_card',
});

// ❌ Ruim
span.addEvent('done');
```

10.3 Tratar Erros Apropriadamente

Sempre registre exceções:

TypeScript

```typescript
try {
  // seu código
} catch (error) {
  span.recordException(error as Error);
  span.setStatus({ code: SpanStatusCode.ERROR });
```

```
  throw error;
}
```

10.4 Usar Contexto Apropriadamente

Mantenha spans com escopo apropriado:

TypeScript

```
// ✅ Bom - Span cobre toda a operação
async createOrder(order) {
  const span = tracer.startSpan('createOrder');
  try {
    // toda a lógica aqui
  } finally {
    span.end();
  }
}

// ❌ Ruim - Span finalizado muito cedo
async createOrder(order) {
  const span = tracer.startSpan('createOrder');
  span.end(); // Finalizado imediatamente!
  // resto da lógica aqui
}
```

---

11. Próximos Passos

11.1 Integração com Banco de Dados

Adicione instrumentação para seu banco de dados:

Bash

```
npm install @opentelemetry/instrumentation-sequelize
# ou
npm install @opentelemetry/instrumentation-mongoose
```

11.2 Integração com APIs Externas

Rastreie chamadas HTTP para serviços externos:

TypeScript

```
const span = tracer.startSpan('call_external_api');
try {
  const response = await fetch('https://api.example.com/data' );
  // OpenTelemetry já instrumenta automaticamente
} finally {
  span.end();
}
```

11.3 Alertas e SLOs

Configure alertas baseados em traces:

- Latência > 1 segundo
- Taxa de erro > 5%
- Spans específicos falhando

11.4 Integração com Ferramentas Comerciais

O OpenTelemetry é agnóstico de backend. Você pode exportar para:

- **Datadog**
- **New Relic**
- **Honeycomb**
- **Elastic APM**
- **AWS X-Ray**

---

12. Conclusão

O OpenTelemetry oferece uma forma padronizada e poderosa de instrumentar aplicações para observabilidade. Ao combinar com RabbitMQ e NestJS, você obtém visibilidade completa sobre como suas mensagens fluem através de seus microsserviços.

Os conceitos apresentados neste material formam a base para construir sistemas observáveis em escala. Com a propagação de contexto automática, você pode rastrear requisições complexas que percorrem múltiplos serviços e identificar gargalos e problemas rapidamente.

Lembre-se de que a observabilidade é um processo contínuo. Comece com os conceitos básicos, implemente gradualmente em seus serviços e refine conforme aprende mais sobre o comportamento de seu sistema.

Referências

[1] OpenTelemetry Documentation -
[2] OpenTelemetry Instrumentation for amqplib -
[3] Jaeger Documentation -
[4] W3C Trace Context -
[5] NestJS Microservices Documentation -
[6] RabbitMQ Documentation -
[7] OpenTelemetry Semantic Conventions for Messaging -
[8] OTEL Collector Documentation -

Versão: 1.0.0

Última atualização: Março de 2026

Autor: Manus AI